



AgoraBot

Navigation sociale pour robot de service

Projet C — IA712 Mobile Robotics

Équipe

Nathan Bouceau · Cora Bouvier · Antoine Durand · Rémi Hamon

Télécom Paris & ENSTA Paris — Juin 2026

Dépôt : github.com/rhmsiatpt2526/agorabot

Table des matières

1. Introduction et contexte.....	3
2. Organisation du projet.....	3
3. Architecture du système	3
3.1 Outils externes et adaptations.....	3
3.2 Fonctionnement du système	5
3.2 Pipeline de costmaps sociales.....	5
3.3 Pipeline de Behavior Tree Social.....	7
4. Lancement, tests et résultats.....	8
5. Difficultés, solutions et leçons apprises.....	8
7. Conclusion et perspectives	9
Annexes	10
Annexe A — Arborescence du dépôt.....	10
Annexe B — Liens vers les dépôts GitHub personnels	10

1. Introduction et contexte

La robotique sociale vise à permettre à un système robotique d'atteindre ses objectifs (e.g. industriels, de service à domicile, en entreprise), tout en évoluant dans un environnement complexe, imprévisible, mouvant et régi par des règles implicites. Les environnements hautement sociaux représentent donc une difficulté supplémentaire pour le robot en termes de déplacement, de localisation, de planification de trajectoire et de prise de décision.

Le projet C de IA712 *Human-aware social navigation* demande à faire déplacer un robot de service dans un environnement de simulation peuplé d'humains dynamiques sur une carte prédéfinie (e.g. maison, entrepôt, etc.). Il doit ainsi naviguer en autonomie d'un point A vers un point B tout en respectant des normes sociales de base : maintenir une distance avec les humains, éviter les collisions, s'arrêter si besoin, modifier son comportement, et trouver des solutions de contournement en cas d'objectifs bloqués.

Dans ce rapport, nous montrerons comment nous sommes parvenus à remplir les exigences du projet à l'aide d'une stack ROS2 modulaire exécutable en une seule commande. Nous présenterons notre organisation de projet, l'architecture de notre système, les difficultés rencontrées et les solutions envisagées, ainsi que les résultats finaux.

2. Organisation du projet

Le projet a pu être mené grâce à l'utilisation de GitHub comme outil de versionnement pour gérer les différentes mises à jour. Le lien du dépôt GitHub du projet se trouve en annexe.

Le projet a pu être mené durant les cinq séances de travail de IA712 organisées les mercredi après-midi à l'ENSTA et à Télécom, ainsi que lors de sessions de travail supplémentaires. Comme nous le détaillerons dans les sections suivantes, l'utilisation de ROS2, le téléchargement des packages requis, et l'exécution du projet sur les différents ordinateurs des membres du groupe ont été accompagnés de nombreux problèmes, bugs, ralentissements et difficultés d'exécutions optimales. Cela s'explique notamment par les systèmes d'exploitation différents (Windows, Linux, MacOS), leurs différentes versions, les ressources de calcul nécessaires et les architectures de calcul des ordinateurs (notamment ARM qui posait des problèmes), et donc les versions de packages à installer.

Les débuts du projet étaient donc laborieux, ce pour quoi nous avons progressivement convergés vers une méthode de travail plus adaptée. Nous réalisions des séances de travail collectives en présentiel, où nous échangeons des idées, discussions des solutions à mettre en place, testions des solutions, puis nous centralisons sur un PC qui parvenait à coder et exécuter les modifications et les ajouts et les déployer sur GitHub dès que possible hors des séances. Cette centralisation a permis de simplifier et d'homogénéiser les modifications du code pour la rendre minimalement fonctionnelle. Un axe d'amélioration du projet consisterait donc en la robustification du projet pour le rendre fonctionnel sur un plus grand nombre d'ordinateurs et d'architectures différentes.

3. Architecture du système

3.1 Outils externes et adaptations

Le projet vient globalement s'inscrire dans un environnement de simulation robotique déjà construit en récupérant des outils classiques et utilisés lors des séances de travaux pratiques de IA712 cette année, tout en les adaptant pour rendre le tout fonctionnel, et surtout en intégrant un module central de gestion de logique sociale, appelé *AgoraBot* comme notre nom de projet.

Nous utilisons ainsi d'abord des outils de l'environnement ROS2 déjà déployés afin de construire notre environnement de simulation sur mesure. Gazebo Classic est employé comme moteur de rendu graphique de l'environnement de simulation. On utilise également Rviz2, outil classique de ROS2 permettant de visualiser les différents éléments techniques liés au robots, tels que les costmaps locales et globales avec leurs différentes couches de coûts. Rviz2 permet également de fixer des objectifs de navigation au robot et de voir ainsi son déplacement en conséquence avec les sorties des planificateurs locaux et globaux (planner / controller). Enfin, Nav2 comme centre de décision du système permet de récupérer les costmaps, le Behavior Tree, les divers paramètres de configurations personnalisés, et de produire une planification et des commandes de contrôle (e.g. `/cmd_vel`) qui seront envoyées au robot pour déplacement.

En dehors des outils « natifs » ROS2, d'autres outils *off-the-shelf* sont récupérés pour permettre la simulation de navigation sociale. Le plus important est HuNavSim (choisi conformément aux recommandations de notre professeur), pour *Human Navigation Simulation*. Cet outil intègre la simulation d'agents (i.e. humains virtuels), la gestion des messages que produisent les humains, leurs comportements, leur rendu, et se branche in fine sur Gazebo. Le comportement des humains repose notamment sur l'outil de modélisation LightSFM, pour *Social Force Model*. Brièvement, les mouvements des humains sont régis selon trois forces, à savoir une force d'attraction envers leur objectif, une force de répulsion interhumains, et une force de répulsion entre humains et obstacles (dont le robot). Cela permet de simuler un comportement réaliste pour les humains, et de veiller à éviter naturellement la trajectoire du robot.

D'autres outils ROS2 secondaires font enfin parti du système global. *People* fournit des messages sur un topic `/people` concernant la position et la vitesse des agents, BehaviorTree.CPP et BehaviorTree.ROS2 fournissent des wrappers ROS2 pour les Behavior Tree (BT) et agorabot_sfm est un petit package « maison » servant à adapter LightSFM au projet. Le principal autre package que l'on a adapté est hunav_gazebo_wrapper, qui fournit un wrapper de Gazebo pour permettre d'y connecter effectivement HuNavSim. Ce wrapper contient notamment des fichiers de lancement adaptés, des maps pour la simulation, des scénarios décrivant le nombre d'agents dans la simulation ainsi que leurs comportements et objectifs individuels. Un certains nombre de modifications ont été effectuées directement dans hunav_gazebo_wrapper pour rendre le projet fonctionnel et par conséquent nous utilisons un fork « maison » avec modifications disponible sur le lien en annexe.

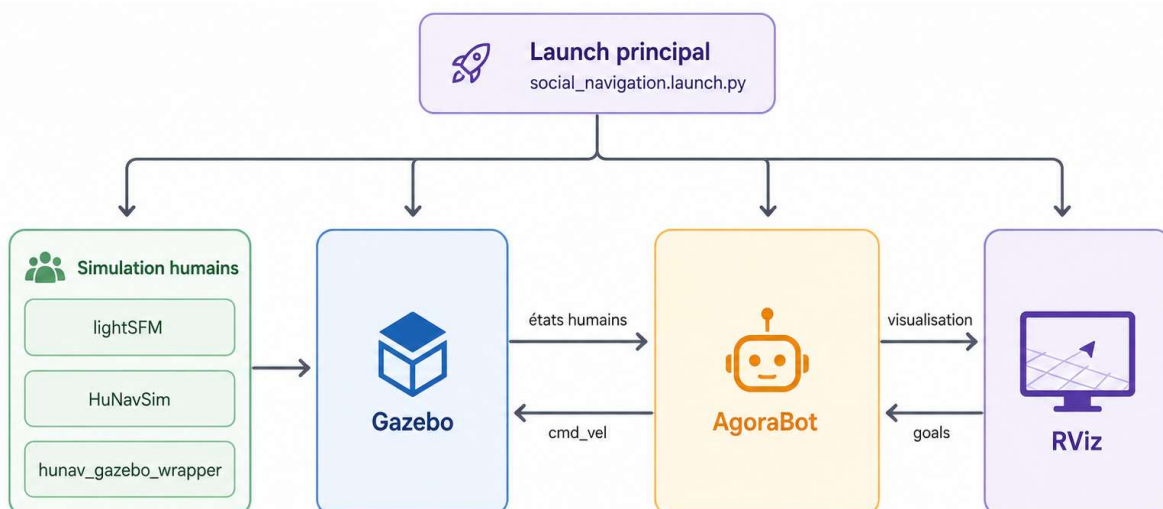


Figure 1 — Architecture simplifiée de l'environnement du projet

3.2 Fonctionnement du système

Le cœur du système, dans les packages *agorabot*, repose sur une logique de calcul de coûts et de décision de comportement cherchant à respecter les normes sociales d'évitement des humains tout en parvenant tout de même à atteindre optimalement les objectifs fixés. Le détail figure en annexe A. La Figure 2 illustre plus précisément son fonctionnement. On commence par venir récupérer les états des humains qui sont publiés par HuNavSim et autres packages annexes mentionnés précédemment, sur lesquels figurent notamment la position et la vitesse. Ces informations sont ensuite traitées par deux processus sociaux en parallèle, i.e. un pipeline de costmaps et un pipeline de BT. Ceux-ci renvoient des informations servant pour Nav2 à calculer des trajectoires socialement adaptées, qui déboucheront sur des commandes robot (i.e. /cmd_vel) récupérées pour faire déplacer le robot sur la map, visible sur Gazebo et Rviz2. Nous détaillons ci-dessous les deux pipelines sociaux.

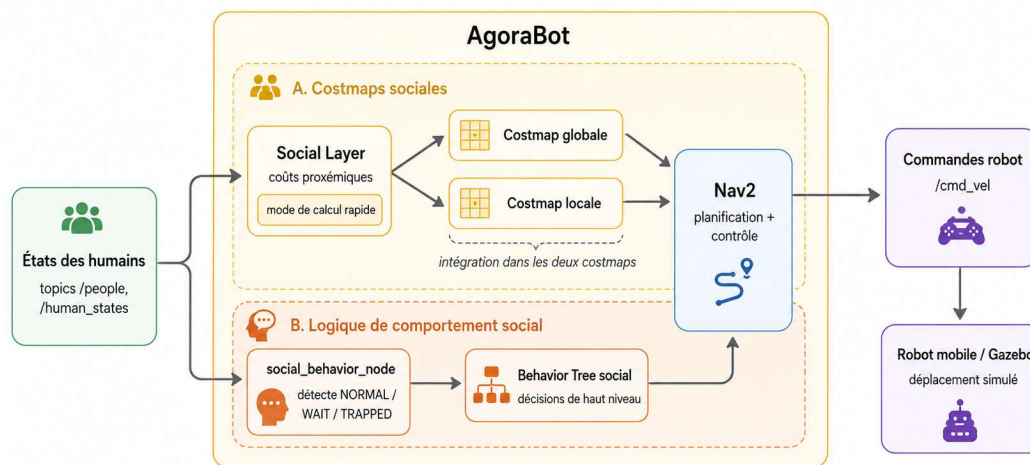


Figure 2 — Vue d'ensemble des processus clés du projet

3.2 Pipeline de costmaps sociales

Ce premier processus que nous avons construit vise à intégrer dans les costmaps des coûts représentant l'exigence d'évitement des humains en déplacement dans la map par le robot. Pour cela, un package C++ *agorabot_social_layer_cpp* a été construit. Par simplicité, nous préférons initialement construire en Python nos divers packages, mais la compatibilité avec le reste de l'architecture nous a amené à utiliser du C++. Il vient récupérer des messages provenant de HuNavSim sur le topic /human_states, contenant la position de chaque humain, et construit une zone de coûts autour de lui, appelée *zone sociale*. Nous avons initialement construit des zones sociales en cercles concentriques conformément à la théorie de proxémie de Hull qui postule des zones intimes, personnelles, sociales, publiques, etc. Puis, nous avons sophistiqué notre approche en rendant les zones non pas rondes mais elliptiques anisotropes en formes d'œufs autour de l'humain, avec une orientation obtenue grâce à la vitesse de l'agent. Cela vient du fait qu'il est plus gênant pour un humain, et donc plus coûteux pour un robot, de passer devant que sur les côtés ou derrière. De plus, nous avons lissé le gradient de coût pour être maximal au centre de l'humain et progressivement décroître jusqu'à atteindre zéro à un mètre environ de l'humain.

Une fois ces coûts construits, nous en tirons avec un taux d'actualisation de 10 Hz une carte de coûts sociaux proxémique, ou *social layer*, qui sera intégrée dans les costmaps de Nav2, ce qui est permis par le fait que notre package de social layer est codé comme un plugin Nav2. Nous intégrons donc ces couches sociales de coûts dans les costmaps locales et globales. Nous voulions en effet initialement ne l'intégrer que dans la local costmap car cela nous paraissait suffisant, mais le robot traversait alors clairement les zones sociales et ne contournait donc pas les humains. Cela s'expliquait par le fait que le planner global ne voyait pas les coûts sociaux, et donc traçait des trajectoires minimales en ligne droite, et par le fait que le local planner n'était pas

assez puissant pour infléchir localement la trajectoire lorsqu'il rentrait dans une zone sociale. Par conséquent les deux costmaps les utilisent. On injecte ces coûts après les couches statiques et d'obstacles, mais avant celle d'inflation, afin de gonfler aussi les obstacles sociaux et respecter la logique d'inflation. Les coûts finaux de la local costmap sont donc repris par le contrôleur local, et ceux de la costmap globale par le planificateur global. Notons enfin que le choix d'utiliser des couches de coûts intégrable dans les costmaps, au lieu de filtres de costmaps appliqués en aval, provient de soucis de simplicité et de moindre besoin de calcul, mais il aurait été probablement faisable de faire avec cette autre approche.

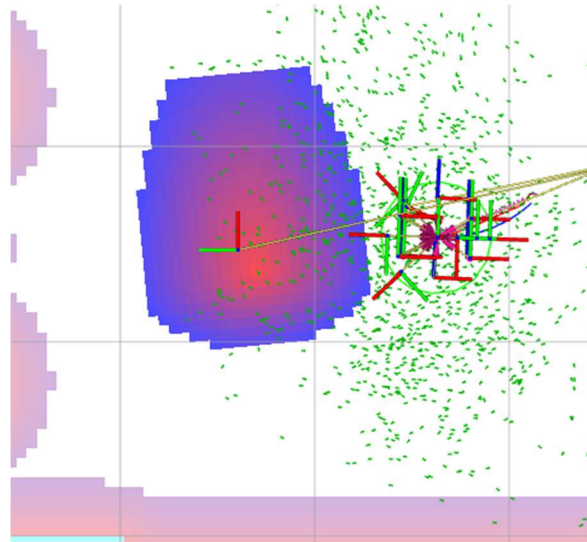


Figure 3 — Modèle de proxémie elliptique anisotrope.

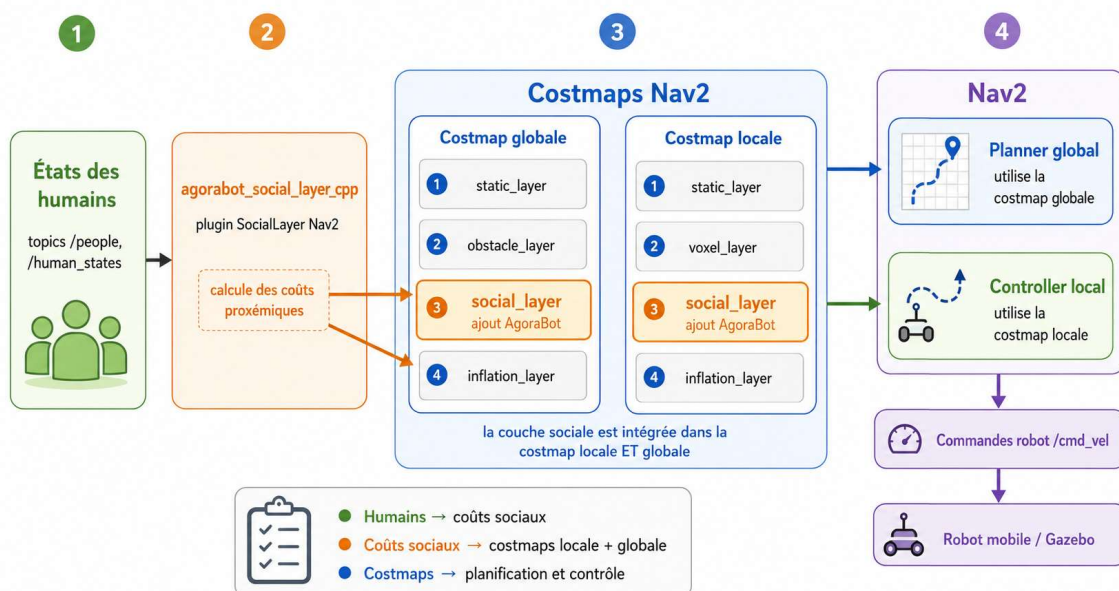


Figure 4 — Détail du pipeline de costmaps sociales

3.3 Pipeline de Behavior Tree Social

En parallèle de cela, un autre processus se met en marche afin de gérer les différents comportements possibles du robot, conformément aux exigences du projet. On souhaite en effet que le robot puisse soit progresser vers son objectif normalement, soit s'arrêter lorsqu'il rencontre un piéton, et avoir une procédure d'urgence en cas de situation de blocage entre un grand nombre d'humains (scénario Trapped). Nous avons donc implémenté un Behavior Tree Social, remplaçant le BT par défaut de Nav2, qui prévoit ces différents comportements. On passe d'abord par un node spécifique, `social_behavior_node`, qui récupère les positions des humains publiés sur le topic `/human_states` et la position du robot publiée sur `/odom`, calcule en conséquence la distance entre le robot et chaque agent, et en déduit le nombre d'agents à moins de 1.2 m de lui. Cela génère donc un nom d'état actuel du robot (actualisé à un taux de 5 Hz), pouvant être `NORMAL`, `WAIT` ou `TRAPPED`. Ce nom d'état est ensuite publié sur un topic construit pour cela, `/social_behavior`.

C'est là qu'un autre dossier de l'arborescence, `behavior_trees/`, contenant un fichier `social_navigation.xml`, vient définir le BT social à proprement parler. Nav2 connaît ce nouveau BT, et vient donc récupérer l'état de `/social_behavior` pour venir naviguer dans l'arbre du BT et donc prendre des décisions. Des nœuds de BT définis dans le dossier `agorobot_bt_nodes` vont préciser les deux états particuliers dans `is_robot_trapped_condition.cpp` et `is_social_wait_condition.cpp`. Remarquons enfin que le BT représente une logique de décision assez simple, et qu'elle pourrait être raffinée avec l'intégration e.g. de modèles de fondation multimodaux poussés.

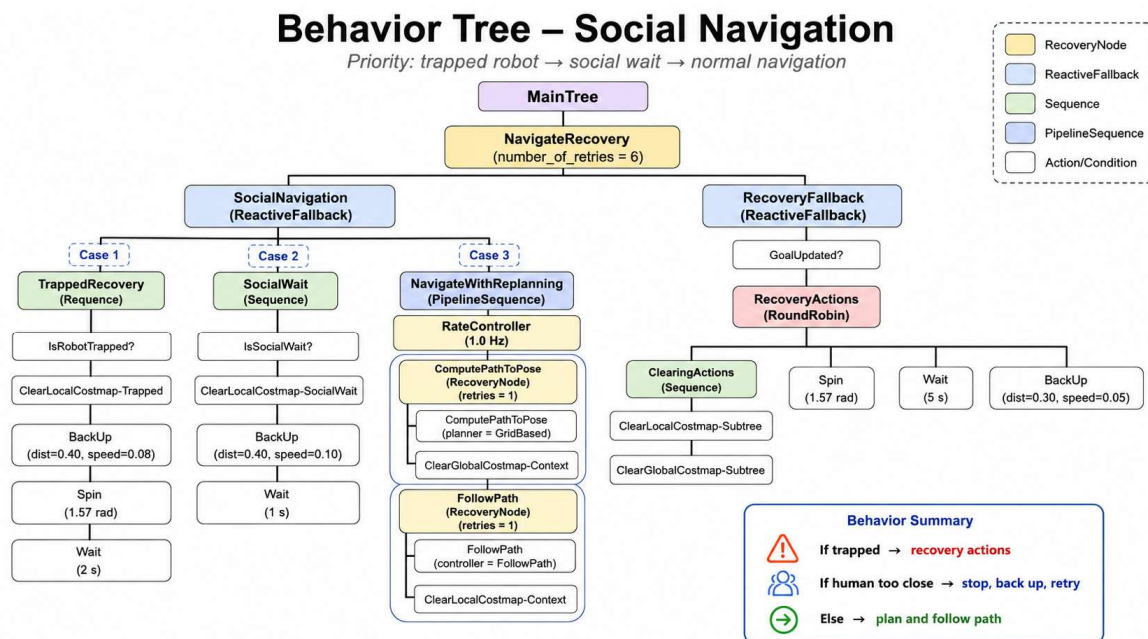


Figure 5 — Behavior Tree social détaillé

Voyons en détail le BT social. Il va tenter six fois une procédure de navigation sociale (branche gauche de la Figure 5), et si ça ne marche pas au bout de six fois il partira sur une branche de fallback, où il nettoie ses costmaps locales et globales, il tourne en rond, attend 5 secondes puis recule un peu, pour ensuite retenter de naviguer. Dans la branche principale du BT, il y a trois cas qui seront testés séquentiellement. D'abord, on vérifie l'état `TRAPPED`, correspondant à la présence d'au moins 3 humains dans l'entourage du robot. Dans ce cas, on enclenche une procédure similaire, avec `local-costmap-clearing`, `backup`, `spin` et `wait 2 s`. S'il n'est pas `TRAPPED`, on vérifie qu'il soit en état de `WAIT` tel que décrit précédemment, et dans ce cas on ne nettoie que la local costmap (inutile de nettoyer aussi la global), il recule puis attend. Enfin, l'état principal et souhaité, `NORMAL`, permet de tenter de naviguer vers l'objectif fixé. Le robot va alors calculer le parcours jusqu'à sa position puis suivre le parcours. Ce BT permet donc de gérer les différents cas requis par les consignes du projet.

4. Lancement, tests et résultats

Le projet permet de lancer l'ensemble de la simulation en une ligne de commande grâce à un *master launch file*, disponible dans le dossier *agorabot_bringup* dans le fichier *social_navigation.launch.py*. Ce fichier permet de conjointement lancer *hunavsim*, *gazebo*, les *wrapper*, les *scenarios*, *agents*, *comportements*, *logique sociale*, *Nav2*, *Rviz*, etc. Le projet s'est concentré sur une démonstration principale, faite dans la map *house* proposée et supportée par *HuNavSim*, avec un scénario de base consistant en 3 humains sur la petite map, avec deux mobiles et un immobile généralement. Des options de lancement ont également été rajoutés : on peut faire apparaître ou non l'interface graphique de *Gazebo* pour économiser des ressources de calcul, on peut faire apparaître 7 agents au lieu de 3 dans un scénario dense, et une possibilité d'autre monde était en cours d'implémentation.

Afin de permettre une démonstration fonctionnelle, il fallait également ajuster les vitesses des robots et des humains. En effet, d'après le modèle *SFM* qui régit le mouvement des humains, la force répulsive anti-obstacles entraîne une fuite des humains à l'approche du robot dans leur voisinage. Par conséquent, nous avons réduit au strict minimum la vitesse des humains, leur permettant de ne pas bouger trop vite à l'approche du robot afin de montrer l'état de *WAIT* déclenché, mais non nulle tout de même afin que l'humain dégage l'espace et laisse le robot atteindre in fine son objectif.

Les tests que nous nous étions fixés ont finalement aboutis à un succès, le robot fixait bien une trajectoire globale contournant l'agent et sa zone sociale (cf. Figure 6), s'arrêtait bien à l'approche de la zone sociale, reculait puis attendait que l'espace se libère, et repassait enfin en zone normale pour atteindre son objectif. Les détails de la démonstration sont visibles dans les vidéos de démonstrations jointes avec le rapport.

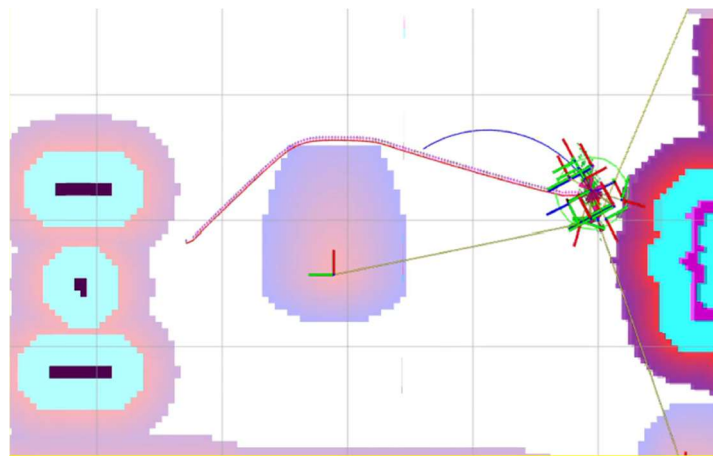


Figure 6 — Illustration de la vidéo de démonstration

5. Difficultés, solutions et leçons apprises

Un certain nombre de difficultés ont été rencontrées durant la réalisation du projet. D'abord, comme il a été mentionné en début de rapport, l'installation et l'exécution, même préliminaire, de *ROS2*, des différents packages d'environnement et du projet lui-même ont été très compliqués, nécessitant des heures d'installation, désinstallation, réinstallation, adaptation et ajustements divers. Un des membres du groupe était pénalisé par son ordinateur Apple, un autre membre avait des difficultés à cause de son architecture ARM, et un troisième rencontrait d'autres difficultés d'installation et de compatibilité bien que sous Windows. Nous avons donc dû utiliser prioritairement un des ordinateurs du groupe sous Windows 11 avec WSL sur lequel le projet fonctionnait mieux pour implémenter les modifications. Ces difficultés nous ont contraint à faire preuve de patience et de résilience mais surtout nous ont apporté une compréhension plus fine des détails du fonctionnement de *ROS2*, qui se sont révélés importants dans la suite.

D'autres difficultés majeures étaient présentes. La stack complète avec tous les nœuds, topics, outils, simulateurs, etc., entraînaient un besoin en ressources de calcul parfois significatif ce qui nous ralentissaient dans l'avancement du projet. De plus, les bugs ou incohérences dans le code étaient rapidement perceptibles lors du lancement car ils pouvaient facilement rendre le chargement de Rviz et Gazebo particulièrement lents. Nous avons été accoutumés à des bugs récurrents, comme la non-apparition d'humains sur Gazebo, leur spawn tous au même endroit que le robot, ou un spawn immobile en position T assez surprenant. Cela nous a donc requis d'être très prudent et de bien monitorer les lancements de simulation, en analysant de nombreux logs de lancements qui étaient également très verbeux. Nous avons été fréquemment surpris de voir qu'une simulation conceptuellement aussi simple pouvait entraîner autant de complexités techniques, mais nous y sommes tout de même parvenus en apprenant. Nous avons donc fini par systématiser l'usage de logs de lancement et de nettoyage de processus et de daemon entre deux runs afin de garantir le succès des simulations.

La mise en place exacte de la logique sociale a été également complexe. Nous avons tenté l'approche de filtres de costmaps comme il était préconisé dans les consignes, avec des *keepout filters*, mais ceux-ci se sont révélés trop compliqués et moins facilement implémentables que la social cost layer décrite précédemment. La connexion entre les logiques sociales et Nav2 était aussi relativement complexe, avec de multiples essais-erreurs. On peut aussi mentionner l'utilisation de multiples dossiers et packages dans ce projet, qui s'est révélé nécessaire alors que nous souhaitions initialement une structure simple. Nous terminons ainsi avec des nœuds Python et C++, des packages off-the-shelf, des packages adaptés, et des packages créés de toute pièce. L'orchestration de l'installation de ce projet avec une telle diversité de sources de code a aussi fait l'objet de différentes tentatives, pour se stabiliser avec l'installation de toutes les dépendances dans un fichier `agorabot.repos` disponible sur le Git principal. Globalement, toutes ces difficultés nous ont montré un ensemble de détails que nous ignorions sur ROS2 et ont donc permis d'améliorer notre compréhension et maîtrise de cet environnement.

7. Conclusion et perspectives

Notre projet AgoraBot fournit in fine une stack ROS 2 complète, fonctionnelle, modulaire et exécutable à l'aide d'un unique master launch file permettant de répondre aux trois exigences du Projet C : création de zones sociales, Behavior Tree de décision face aux piétons, et comportements de récupération. La navigation sociale est obtenue par une carte de coûts sociaux en proxémie elliptique anisotrope intégrée dans une couche sociale des costmaps locales et globales de Nav2, ainsi qu'un Behavior Tree social fonctionnant en parallèle en détectant les états NORMAL, WAIT et TRAPPED et actionnant si besoin des comportements anti-blocage. Le tout permet de réaliser avec succès une démonstration de navigation sociale en évitant les humains et en contournant leurs zones sociales tout en respectant les règles d'attentes et de récupération imposées.

Les améliorations futures et perspectives de notre travail pourraient inclure différents éléments. D'abord, la variation des scénarios de lancement, qui a déjà été explorée mais plutôt freinée par manque de ressources de calcul : scénario *dense* avec multiples agents, mais aussi *trapped* pour volontairement enfermer le robot et tester le mode TRAPPED, ou *agressive* pour simuler des humains venant présenter un comportement plus agressif envers le robot. On pourrait également intégrer un autre contrôleur Nav2 au projet, qui utilise pour l'instant DWB (MPPI ne fonctionnait pas lors de nos tests) afin d'améliorer la navigation. On pourrait également sophisticationner les démonstrations en ajoutant et mesurant des métriques aux démonstrations. Le BT pourrait également être plus sophistiqué avec plus d'états et l'on pourrait tenter de mieux régler les paramètres de simulation. On pourrait enfin accélérer les humains et le robot pour rendre la démonstration plus réaliste et intéressante, afin de produire d'autres vidéos de démonstration. D'autres améliorations sont évidemment envisageables.

Annexes

Annexe A — Arborescence du dépôt

